



**College of Computing and Informatics
Department of Computer Science**

Course Syllabus

Course Title Programming II

**Course Code
1501211**

Spring 2025/2026

Course Syllabus

Course Title	Programming II				
Course Code	1501211				
Credit Hours	3	Contact Hours	4		
Course Category	Program required				
Course Type	Compulsory				
Pre-requisite(s)	1411115 - Introduction to Computer Science I OR 1411116 - Programming I OR 1501116 - Programming I OR 1501115 - Intro. to Computer Science I				
Co-requisite(s)					
Semester	Spring 2025/2026				
Instructor Name	Email	Classroom	Section	Session Type	Time
Menen Kassim M. Mohammed Abdulwahab	mabdulwahab@sharjah.ac.ae	A3-COMP011	02Y	Laboratory	R 1400 - 1550
Naveed Ahmed	nahmed@sharjah.ac.ae	C9-COM010	32Y	Laboratory	W 1400 - 1550
Isam Mashhour Hasan Al Jawarneh	ijawarneh@sharjah.ac.ae	A3-179	02	Lecture Undergraduate	MW 0930 - 1045
Menen Kassim M. Mohammed Abdulwahab	mabdulwahab@sharjah.ac.ae	A3-COM112	01X	Laboratory	M 1400 - 1550
Danilo Dessi	ddessi@sharjah.ac.ae	A10-112	01	Lecture Undergraduate	TR 0930 - 1045
Naveed Ahmed	nahmed@sharjah.ac.ae	C3-COMP114	31X	Laboratory	T 1400 - 1550
Muhammad Anwar Shahid	mshahid@sharjah.ac.ae	C3-COMP011	31Y	Laboratory	W 1400 - 1550
Muhammad Anwar Shahid	mshahid@sharjah.ac.ae	C9-COM005	32X	Laboratory	M 1400 - 1550
Muhammad Anwar Shahid	mshahid@sharjah.ac.ae	C10-102	32	Lecture Undergraduate	MW 1100 - 1215
Muhammad Anwar Shahid	mshahid@sharjah.ac.ae	C10-101	31	Lecture Undergraduate	TR 0930 - 1045
Ayesha Muhammad Talha	atalha@sharjah.ac.ae	A3-COM112	02X	Laboratory	W 1400 - 1550
Danilo Dessi	ddessi@sharjah.ac.ae	A3-COMP011	01Y	Laboratory	T 1400 - 1550

Course Description:

This course introduces fundamental conceptual tools and their implementation of object-oriented design and programming, such as object, type, class, implementation hiding, inheritance, parametric typing, function

overloading, polymorphism, source code reusability, and object code reusability. Object-Oriented Analysis/Design for problem solving. Implementation of the Object-Oriented programming paradigm is illustrated by program development in an OO language (C++).

Course Objectives/Goals:

The goals of this Course are to:

- Provide students with concepts of object-oriented programming
- Familiarity with the use of object-oriented programming paradigm in problem solving
- Familiarity with pointers, memory management, and parameter passing in C++
- Know and appreciate the use of object-oriented programming in software development: information hiding, code reuse, software customization, etc

Course Learning Outcomes:

By the end of successful completion of this Course, the student will be able to:

1. Analyze problems and implement the corresponding object-oriented programming solution
2. Use information hiding, templates, and software reuse effectively in software development
3. Develop, test, and debug large solutions using Object Oriented Programming (OOP)
4. Analyze dynamic binding and abstract classes in software development
5. Apply overloading and templates in software development

Alignment of Course Student Learning Outcomes to Program Student Learning Outcomes

Program Learning Outcomes (PLOs)	Course Learning Outcomes (CLOs)
Design, implement, and evaluate a computing-based solution to meet a given set of computing requirements in the context of the program's discipline.	- Use information hiding, templates, and software reuse effectively in software development - Develop, test, and debug large solutions using Object Oriented Programming (OOP)
Apply computer science theory and software development fundamentals to produce computing-based solutions.	- Analyze dynamic binding and abstract classes in software development - Apply overloading and templates in software development
Analyze a complex computing problem and apply principles of computing and other relevant disciplines to identify solutions.	- Analyze problems and implement the corresponding object-oriented programming solution - Apply overloading and templates in software development
Identify, formulate, and solve complex computer engineering problems by applying principles of engineering, science, and mathematics	- Analyze problems and implement the corresponding object-oriented programming solution - Apply overloading and templates in software development
Apply engineering design to produce solutions that meet specified needs with consideration of public health, safety, and welfare, as well as global, cultural, social, environmental, and economic factors	- Analyze dynamic binding and abstract classes in software development - Apply overloading and templates in software development
Develop and conduct appropriate experimentation,	

Program Learning Outcomes (PLOs)	Course Learning Outcomes (CLOs)
analyze and interpret data, and use engineering judgment to draw conclusions	- Use information hiding, templates, and software reuse effectively in software development - Develop, test, and debug large solutions using Object Oriented Programming (OOP)
Identify, formulate, and solve complex problems in cybersecurity engineering or related fields by applying principles of engineering, science, and mathematics.	- Analyze problems and implement the corresponding object-oriented programming solution - Apply overloading and templates in software development
Apply engineering design to produce solutions that meet specified needs with consideration of public health, safety, and welfare, as well as global, cultural, social, environmental, and economic factors.	- Analyze dynamic binding and abstract classes in software development - Apply overloading and templates in software development
Conduct appropriate experimentation, analyze and interpret data, and use engineering judgment to draw conclusions.	- Use information hiding, templates, and software reuse effectively in software development - Develop, test, and debug large solutions using Object Oriented Programming (OOP)
Design and evaluate a software system, process, or program to meet desired needs.	- Use information hiding, templates, and software reuse effectively in software development - Develop, test, and debug large solutions using Object Oriented Programming (OOP)
Apply design and development principles in the construction of software and informatics- based solutions of varying complexity.	- Analyze dynamic binding and abstract classes in software development - Apply overloading and templates in software development
Analyze a problem, identify and define the computing requirements appropriate to its solution.	- Analyze problems and implement the corresponding object-oriented programming solution - Apply overloading and templates in software development

Weekly Distribution of Course Topics/Contents

Week	Topic	Comments*	CLOs
1	- Introduction: Object-based programming (additional notes). - Records (Structs): definition, operations on a struct, data manipulation using a struct, relationship between a struct and functions, arrays vs. struct, arrays in structs, structs in arrays, structs within structs.	-Introduce the students to the structs.	1
2	Classes and Data Abstraction: definition of a class; private, protected, and public members; class implementation; Objects declaration; Accessing class members; Scope resolution operator; Accessors and mutator functions.	- Introduce the students to the classes and resolve the same problem using a class instead of a struct. - Control the security of	1,2

Week	Topic	Comments*	CLOs
		information using the member access specifiers. - Classes with static members.	
3	Constructors and destructors; Constructor with default parameters; Abstract Data Types (ADTs); Data abstraction and information hiding; Project definition.	- Introduce the students to the classes and resolve the same problem using a class instead of a struct. Control the security of information using the member access specifiers. - Classes with static members	1,2
4	Static members of a class; Applications of static members of a class.	- Introduce the students to the classes and resolve the same problem using a class instead of a struct. Control the security of information using the member access specifiers. - Classes with static members	1,2
5	Inheritance and Composition: Derived and base classes; Base class member function redefinition (overloading and overriding).	- Introduce the students to inheritance. - Introduce the students to composition.	2,3,4
6	Base and derived class constructors; Derived class header file; Public, protected, and private inheritance.	- Introduce the students to inheritance. - Introduce the students to composition.	2,3,4
7	Composition; Application of inheritance and composition.	- Introduce the students to inheritance. - Introduce the students to composition.	2,3,4
8	Pointers, and Classes: Pointer data type and pointer variables; Address (&) and the dereferencing operators (*); dynamic variables; dynamic memory management with the new and delete operators. Dynamic arrays; Classes with pointer data members; Memory leak; Dangling pointers; Shallow copy and deep copy; Copy constructor	-Using pointers as data members of a class. -Dynamic memory allocation. -Introduce the students to virtual functions. - Introduce the students to polymorphism.	2,3,4
9	Virtual functions: Static binding and dynamic binding; Pure virtual functions; Abstract base classes; Virtual destructors.	-Using pointers as data members of a class. -Dynamic memory allocation. -Introduce the students to virtual functions. - Introduce the students to polymorphism.	3,4
10	Polymorphism; Application of polymorphism.	-Using pointers as data members of a class. -Dynamic memory allocation. -Introduce the students to virtual functions.	3,4

Week	Topic	Comments*	CLOs
		- Introduce the students to polymorphism.	
11	Overloading and Templates: Operator overloading; Operator functions; Restrictions on operator overloading; Friend functions; Members and non-members of a class.	- Introduce the students to operator overloading. - Overloading different operators. - Introduce the students to templates.	3,4
12	Overloading binary operators; Overloading the assignment operator =; Overloading the extraction >> and insertion << operators; Overloading the increment and decrement operators; Other operators; this pointer; Cascading.	- Introduce the students to operator overloading. - Overloading different operators. - Introduce the students to templates.	5
13	Function templates; class templates	- Introduce the students to operator overloading. - Overloading different operators. - Introduce the students to templates.	5
14	Exception Handling: Definition of an exception; handling exceptions in a program; handling exceptions using try / catch; C++ exception classes; creating exception classes; throwing and re-throwing exceptions.	Introduce the students to exception mechanize in C++.	5
15	Final Exam		3

Scheduling of laboratory and other non-lecture sessions, as appropriate

Week	Topic	Comments
2	Lab #1 Structs	Define a struct and functions with struct variable parameters.
3	Lab#2 Classes	Define a class with member functions and data members.
4	Lab#3 Data Abstraction	Define a class with constructors, and enforce the principle of least privileged.
5	Lab#4 Static members	Define a class with static data members and member functions.
6	Lab#5 Inheritance	Define classes in an inheritance hierarchy.
7	Lab#6 Inheritance	More on inheritance.
8	Lab#7 Composition	Define classes with composition.
9	Lab#8 Pointers	Define a class with a pointer data member
10	Lab#9 Virtual functions	Virtual functions in an inheritance hierarchy.

Week	Topic	Comments
11	Lab#10 Overloading	Operator overloading.
12	Lab#11 Overloading	More on operator overloading.
13	Lab#12 Templates	Class templates.

Information on out-of-class assignments with due dates for submission

Assignment/Activity	Due Date	Comments
Assignment 1	Weeks: 5 - 8	Define a hierarchy of classes using inheritance and composition.
Assignment 2	Weeks: 8 - 12	Implement the concept of polymorphism.

Students' Assessment:

Students are assessed as follows:

Assessment Tool(s)**	Date	Weight (%)
Lab practical exercises	Throughout the semester	10
Assignments	Throughout the semester	5
Midterm Exam	Midterm exams period	25
Quizzes	Throughout the semester	15
Final Exam	Final exams period	45
Total		100

Course Outcome Assessment Plan:

Course LOs	Teaching/Learning Method(s)	Assessment Tool(s)	Performance Indicators	UoS competencies	Other competencies
Analyze problems and implement the corresponding object-oriented programming solution	- Lab session - Lectures - Problem Solving	- Assignments - Exams - Lab Exercises	Course assessment Student marks 60%	- Critical and analytical thinking - Problem solving	
Use information hiding, templates, and software reuse effectively in software development	- Lab session - Lectures - Problem Solving	- Assignments - Exams - Lab Exercises	Course assessment Student marks 60%	- Critical and analytical thinking - Problem solving	
Develop, test, and debug large solutions using Object Oriented Programming (OOP)	- Lab session - Lectures - Problem Solving	- Assignments - Exams - Lab Exercises	Course assessment Student marks 60%	- Critical and analytical thinking - Problem solving	
Analyze dynamic binding and abstract classes in	- Lab session	- Assignments	Course assessment	- Critical and	

Course LOs	Teaching/Learning Method(s)	Assessment Tool(s)	Performance Indicators	UoS competencies	Other competencies
software development	- Lectures - Problem Solving	- Exams - Lab Exercises	Student marks 60%	analytical thinking - Problem solving	
Apply overloading and templates in software development	- Lab session - Lectures - Problem Solving	- Assignments - Exams - Lab Exercises	Course assessment Student marks 60%	- Critical and analytical thinking - Problem solving	

Scope and Nature of Assessment Tools:

Weekly lab exercises:

Students are expected to attend and complete the lab exercises during the lab. The submission for this assignment is the .cpp and .h file. You should export this file and upload it via the link in the Blackboard — instructions can be found on the assessment submission page. Do not submit your solutions in any other format, as they will not be graded. Labs will be graded, and feedback will be shared with students on the Blackboard.

Assignments:

Students will be given a homework assignment and have to complete it within a timeframe. The submission for this assignment is the .cpp and .h file. You should export this file and upload it via the link in the Blackboard — instructions can be found on the assessment submission page. Do not submit your solutions in any other format, as they will not be graded. Homework assignments will be graded, and feedback will be shared with students on the Blackboard.

Midterm:

This assessment aims to put into practice the programming skills you have learned via the topic content and lab sessions. You will be given the opportunity to demonstrate your ability to analyse problems, apply basic programming constructs to design computational solutions and implement executable code in the C++ programming language. The midterm will be graded, and feedback will be shared with students.

Quizzes:

Students are expected to attend and take a quiz during the lecture. The quiz will be graded, and feedback will be shared with students.

Final Exam:

This assessment aims to put into practice the programming skills you have learned via the topic content and lab sessions. You will be given the opportunity to demonstrate your ability to analyse problems, apply basic programming constructs to design computational solutions and implement executable code in the C++ programming language. The final exam will be graded, and feedback will be shared with students.

Teaching and Learning Resources:

Textbook(s):

Malik, D. S. (2017). C++ Programming: From Problem Analysis to Program Design. 8th ed. -. ISBN:978-133711756.

Other Resources:

Any available textbooks in the library about Object Oriented Programming in C++.

Attendance policy:

Attendance is compulsory. A student missing 10% of the total allocated course hours will receive 1st warning notice and a student missing 15% will receive 2nd warning notice. A student missing 20% will be forced to withdraw (in accordance with the university regulations).

Plagiarism/Cheating:

Students are expected to do their own work. You are allowed to work on assignments in teams only if specified by the instructor. In other words, students are encouraged to communicate about general principles of the course, but all assigned homework must be done on an individual basis. The instructor is available to provide any assistance that you may need. Cheating is considered a serious offense by the university. You should be aware of the severe penalty for cheating (refer to the student code of conduct published in the university catalogue).

Notes:

Students are expected to attend weekly labs to practice main concepts of the course. Students are given programming assignments too. Each assignment is usually made of several problems.

Grading rubrics:

Grading Rubric for Course Assignments					
Scaled Score:		0/1	2	3	4
Criteria	Weight	Below Standard	Approaching Standard	At Standard	Exceeds Standard
Focus: Purpose	10%	No awareness	Shows limited awareness of purpose.	Shows awareness of purpose.	Purpose is clear.
Main idea	10%	No main idea	Vague sense of a main idea, weakly supported throughout the paper.	There is a main idea supported throughout most of the paper.	Clearly presents a main idea and supports it throughout the paper.
Organization: Overall	5%	No sense of organization	There is a sense of organization, although some of the organizational tools are used weakly or missing.	Good overall organization, includes the main organizational tools.	Well-planned and well-thought out. Includes title, introduction, statement of main idea, transitions, and conclusion.

Grading Rubric for Course Assignments					
Scaled Score:		0/1	2	3	4
Organization: Paragraphs	5%	Paragraphs lack clear ideas	Some paragraphs have clear ideas, support from examples may be missing and transitions are weak.	Most paragraphs have clear ideas, are supported with some examples and have transitions.	All paragraphs have clear ideas, are supported with examples and have smooth transitions.
Content	50-60%	Content is not sound	Content is sound and solid; ideas are present but not particularly developed or supported; some evidence, but usually of a generalized nature.	Well-presented and argued; ideas are detailed, developed and supported with evidence and details, mostly specific.	Exceptionally well-presented and argued; ideas are detailed, well-developed, supported with specific evidence and facts, as well as examples and specific details.
Research (if assignment includes a research component)	10%	The paper does not use adequate research or if it does, the sources are not integrated well. They are not cited correctly according to MLA style, nor listed correctly on the Works Cited page.	Sources support some claims made in the paper, but might not be integrated well within the paper's argument. There may be a few errors in MLA style.	Sources are well integrated and support the paper's claims. There may be occasional errors, but the sources and Works Cited conform to MLA style sheet.	Sources are exceptionally well-integrated and they support claims argued in the paper very effectively. Quotations and Works Cited conform to MLA style sheet.
Style: Sentence structure, word, choice, tone	5%	Sentences aren't clear	Sentences are generally clear word usage generally appropriate, details are somewhat adequate but may have awkward structure or unclear content; there may be patterns of punctuation errors.	Sentences are clear, word usage appropriate, details are enough but may lack variation; a few may be awkward and there may be a few punctuation errors.	Sentences are clear, word usage appropriate; details are enough and varied in pattern, from simple to complex, with excellent use of punctuation.

Grading Rubric for Course Assignments					
Scaled Score:		0/1	2	3	4
Grammar and Mechanics	5%	Continuous errors	Shows a pattern of errors in spelling, grammar, syntax and/or punctuation. Could also be a sign of lack of proof-reading.	A few errors in grammar, spelling, syntax and punctuation, but not many.	Excellent grammar, spelling, syntax and punctuation.